

CyberGuard

Phase III: Data Management and Integration Layer`nPrepared for Assignment 4
Presentation

1. Assignment Goal and Scope

- Extend the existing CyberGuard project with a modern data management layer.
- Cover all required topics: data access, databases, APIs, async processing, caching, security, transformation, and monitoring.
- Use modular backend design with clear separation of controller, service, and data layers.
- Provide a practical implementation that can be demonstrated live during class.

2. Project Evolution Across Phases

- Phase I: React-based cybersecurity website and dashboard user interface.
- Phase II: Enterprise integration layer with orchestration, messaging, correlation, and fault handling on port 3001.
- Phase III: Data management backend with databases, APIs, caching, security, and monitoring on port 3002.
- Separate Phase II and Phase III servers were kept intentionally to avoid regressions and show layered evolution.

3. Final Architecture Overview

- Frontend: React + Vite + TypeScript + Tailwind UI.
- Phase III backend: Express.js server exposing /api/v3 REST endpoints.
- Modular backend flow: routes -> controllers -> services -> models -> databases.
- Support services: cache manager, async processor, data transformer, logger, and security middleware.

4. Data Access Layer Implementation

- Controllers remain thin and delegate work to services such as authService, incidentService, threatIntelService, and systemService.
- Services contain validation flow, caching decisions, audit actions, async job triggering, and transformations.
- Models isolate database operations for SQLite and NeDB.
- This design satisfies the assignment requirement for modular CRUD with proper separation of concerns.

5. Database Integration: SQL + NoSQL

- Relational database: SQLite using sql.js for users, incidents, threat signatures, incident relations, and audit logs.
- Non-relational database: NeDB for threat intelligence, system events, scan results, and persistent job queues.
- Indexes were added for efficient filtering and lookup.
- Using both SQL and NoSQL increases the academic value of the solution.

6. RESTful API-Based Data Communication

- Authentication endpoints: register and login with JWT token generation.
- Incident endpoints: full CRUD, statistics, XML export, external format export, and STIX-like export.
- Threat intelligence endpoints: full CRUD, search, and XML export.
- System endpoints: health, metrics, cache stats, async stats, audit log, and XML-to-JSON conversion.

7. Asynchronous Data Processing

- AsyncProcessor implements producer-consumer behavior using an event-driven model.
- Jobs are persisted in the NeDB jobQueue collection so they survive downtime.
- Built-in handlers process threat analysis, scan processing, reporting, notification, and data cleanup.
- Offline simulation endpoint demonstrates that queued messages can be processed later.

8. Caching Strategy and Performance

- In-memory cache stores frequently accessed incident and threat-intel responses.
- Absolute TTL and sliding expiration policies are used depending on the data type.
- Tag-based invalidation clears related cached records after create, update, or delete operations.
- A benchmark endpoint compares uncached and cached access times to show performance improvement.

9. Data Security and Validation

- JWT-based authentication secures protected routes.
- bcrypt password hashing protects stored credentials.
- Helmet, rate limiting, input sanitization, and express-validator reduce common vulnerabilities.
- Role-based access control differentiates admin, analyst, and viewer permissions.

10. Data Transformation and Interoperability

- Incident data can be restructured into an external JSON format for partner systems.
- JSON-to-XML and XML-to-JSON conversions support heterogeneous system communication.
- Incidents can be exported in a STIX-like threat intelligence format.
- This demonstrates interoperability in distributed applications, not only local storage.

11. Logging and Monitoring

- Winston provides structured logging to console and log files.
- Request logging tracks path, status code, duration, and request identifiers.
- Metrics endpoint exposes request, response-time, database, and cache statistics.
- Audit log and system-events endpoints support debugging, accountability, and maintenance.

12. Live Demonstration Flow

- Start frontend, Phase II backend, and Phase III backend.
- Open the /data-management dashboard in the React application.
- Login as admin, create a new incident, and trigger a background job.
- Run the cache benchmark and XML-to-JSON transformation to show advanced features live.

13. Challenges, Fixes, and Lessons Learned

- The initial scaffold needed a stronger service layer and cleaner separation of concerns.
- Database startup ordering had to be fixed so SQLite initializes before seeding and serving requests.
- Seed logic was improved to avoid duplicating NoSQL data on every restart.
- A dedicated Phase III dashboard was added to make demonstration and explanation easier.

14. Conclusion and Future Improvements

- CyberGuard Phase III now satisfies the assignment requirements with a working end-to-end implementation.
- The project demonstrates real CRUD, dual-database design, secure APIs, async jobs, caching, transformation, and observability.
- Future improvements could include Redis, RabbitMQ, PostgreSQL, Docker deployment, and merged server orchestration.
- Current result: a complete, testable, and presentation-ready data management layer.